

Vehicle Parking App - V1: Project Report

Modern Application Development I

Author

Name: Prateek Sharma

Roll Number: 24dp1000047

Email: 24dp1000047@ds.study.iitm.ac.in

Passionate developer focused on crafting interactive, user-centric web apps.

Description

Vehicle Parking App – V1 is an interactive, multi-user application designed to manage parking operations for 4-wheelers. It supports two user roles: **administrator** and **regular users**, and allows efficient management of **parking lots**, **individual parking spots**, and **parked vehicles**.

Technologies Used

1. **Flask**: Lightweight Python web framework for routing and backend logic.
2. **Jinja2**: Templating engine for rendering dynamic HTML pages.
3. **SQLite**: Relational database for storing application data.
4. **HTML/CSS**: Front-end technologies for responsive design.
5. **Date Time**: Used for updating time and date in the database
6. **Flask Extensions**:
 - **Flask-SQLAlchemy**: ORM for database interaction.

AI-Percentage

I used AI in Backend(flask+DB) percentage: 10% and CURD operation-5%.

DB Schema Design

1. User

- **Columns**: id, email, password, fullname, address, pin_code, role
 - **Constraints**: id is the primary key, email must be unique
-

2. ParkingLot

- **Columns**: id, prime_location_name, pin_code, price, maximum_number_of_spots,
 - **Constraints**: id is the primary key, each parking lot, identified by a unique id as the primary key, have multiple associated parking spots.
-

3. ParkingSpot

- **Columns**: id, lot_id, spot_number, is_occupied.
 - **Constraints**: id is the primary, lot_id is a **foreign key** referencing ParkingLot(id)
-

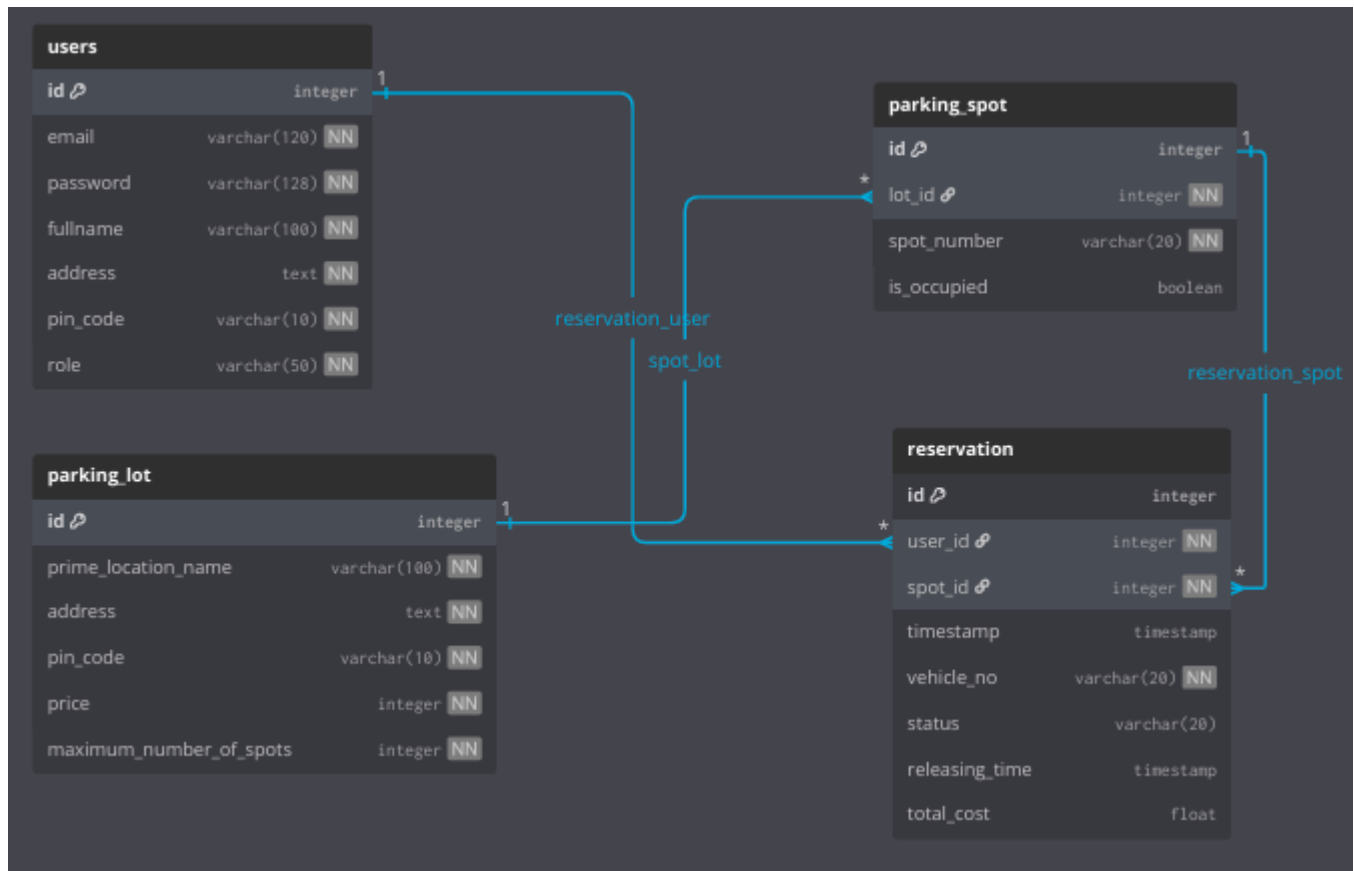
4. Reservation

- **Columns**: id, spot_id, user_id, timestamp, , vehicle_no, status releasing_time,
 - **Constraints**: id is the primary key, spot_id is a foreign key linking to parkingSpot(id), vehicle_no must be unique, status must be either 'booked' or 'released'.
-

Reasoning:

The schema ensures normalized design with proper foreign key relationships. Tracks which user reserves which spot, when, and for how long.

ER Diagram



Architecture

The project follows an MVC (Model-View-Controller) architecture:

1. **Controllers (routes.py)**: Handle business logic and routing.
2. **Models (models.py)**: Define database schema using SQLAlchemy ORM.
3. **Templates (templates/)**: Jinja2 templates render dynamic HTML pages.
4. **Static Files (static/)**: Contains CSS files for styling and interactivity.
5. **Main File (app.py)**: Contains the app initialization and database creation code.

Feature

The platform will have **two** roles:

1. **Admin - root access** - It is the superuser of the app and requires no registration
 - Admin is also known as the **superuser**
 - Admin can create a new parking lot
 - Each parking lot can have any number of parking spots for 4-wheeler parking
 - Each parking lot can have a different price
 - Admin can view the status of all available parking spots on his/her dashboard

- Admin can edit/delete any number of parking lots, i.e., admin can increase or decrease the number of parking spots inside the lot.

2. User - Can reserve a parking space

- Register/Login
- Choose an available parking lot
- Book the spot (automatically allotted by the app after booking)
- Release or vacate the spot.

Demo-Video link:-

https://drive.google.com/file/d/1XesM62wP3076DaOyNS-dlzK3bY5NIV0_/view?usp=sharing